

Лабораторная работа №2

Первоначальная настройка Git

Mikhail Vzvarov (vzvarovmichail)

2026-08-27

Лабораторная работа №2

Первоначальная настройка Git

Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с Git.

Задание

1. Установить Git и GitHub CLI
 2. Выполнить базовую настройку Git
 3. Создать SSH-ключи (RSA и Ed25519)
 4. Создать PGP-ключ
 5. Зарегистрироваться на GitHub
 6. Добавить SSH-ключ на GitHub
 7. Добавить PGP-ключ на GitHub
 8. Настроить автоматическую подпись коммитов
 9. Создать рабочее пространство для курса
 10. Загрузить структуру курса на GitHub
-

Выполнение работы

1. Установка Git и gh `'''bash sudo dnf install git sudo dnf install gh ##3.`
Базовая настройка Git

Команды: bash

```
git config --global user.name "Mikhail Vzvarov" git config --global user.email
"1032251972@rudn.ru" git config --global core.quotepath false git config --global
init.defaultBranch master git config --global core.autocrlf input git config --global
core.safecrlf warn
```

```
##Проверка: bash
```

```
git config --list
```

```
https://images/git-config.png ##4. Создание SSH-ключей
```

```
RSA (4096 бит): bash
```

```
ssh-keygen -t rsa -b 4096
```

```
Ed25519: bash
```

```
ssh-keygen -t ed25519
```

```
##Проверка: bash
```

```
ls -la ~/.ssh/
```

```
https://images/ssh-keys.png ##5. Создание PGP-ключа
```

```
Команда: bash
```

```
gpg --full-generate-key
```

```
##Параметры:
```

```
Тип: RSA and RSA
```

```
Размер: 4096
```

```
Срок действия: 0 (бессрочный)
```

```
Имя: Mikhail Vzvarov
```

```
Email: 1032251972@rudn.ru
```

```
##Проверка: bash
```

```
gpg --list-keys
```

```
https://images/pgp-key.png ##6. Добавление SSH-ключа на GitHub
```

```
Копируем ключ: bash
```

```
cat ~/.ssh/id_ed25519.pub
```

```
##Действие на GitHub:
```

```
Settings → SSH and GPG keys → New SSH Key
```

```
Title: Work PC
```

```
Key: вставить содержимое ключа
```

```
https://images/github-ssh.png ##7. Добавление PGP-ключа на GitHub
```

Экспортируем ключ: bash

```
gpg --armor --export (key)
```

Действие на GitHub:

Settings → SSH and GPG keys → New GPG Key

Title: GPG-key

Key: вставить экспортированный ключ

<https://images/github-gpg.png> ##8. Настройка подписи коммитов

Команды: bash

```
git config --global user.signingkey git config --global commit.gpgsign true git config --global gpg.program $(which gpg2)
```

Проверка: bash

```
git config --list | grep gpg
```

<https://images/gpg-sign.png> ##9. Авторизация в gh

Команда: bash

```
gh auth login
```

<https://images/gh-auth.png> ##10. Создание рабочего пространства

Создаём структуру каталогов: bash

```
mkdir -p ~/work/study/2025-2026/"Операционные системы" cd ~/work/study/2025-2026/"Операционные системы"
```

Создаём репозиторий на GitHub: bash

```
gh repo create study_2025-2026_os-intro --template=yamadharma/course-directory-student-template --public
```

<https://images/repo-create.png>

Клонируем репозиторий: bash

```
git clone --recursive git@github.com:vzvarovmikhail/study_2025-2026_os-intro.git os-intro
```

##11. Настройка каталога курса bash

```
cd os-intro rm package.json echo os-intro > COURSE make
```

<https://images/course-structure.png> ##12. Отправка на GitHub bash

```
git add . git commit -am 'feat(main): make course structure' git push
```

<https://images/git-push.png> Результат

Репозиторий на GitHub: text

https://github.com/vzvarovmikhail/study_2025-2026_os-intro

<https://images/repo-final.png> ## Домашнее задание ## Контрольные вопросы

###1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий (VCS) — это инструменты для управления изменениями в файлах. Они предназначены для:

Хранения истории изменений

Возможности отката к предыдущим версиям

Совместной работы нескольких разработчиков

Разрешения конфликтов при слиянии

###2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

Хранилище (Repository) — база данных, содержащая всю историю изменений проекта

Commit — фиксация изменений в хранилище с комментарием

История — цепочка коммитов, показывающая эволюцию проекта

Рабочая копия — локальная версия файлов, с которой работает разработчик

###3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS?

Централизованные (CVS, Subversion):

Есть единый центральный сервер

Все изменения отправляются на сервер

Без сервера нельзя работать с историей

Децентрализованные (Git, Mercurial):

Каждый разработчик имеет полную копию репозитория

Можно работать офлайн

Изменения синхронизируются позже

###4. Опишите действия с VCS при единоличной работе с хранилищем.

Создать репозиторий: `git init`

Добавлять изменения: `git add`

Фиксировать изменения: `git commit`

Просматривать историю: `git log`

Откатываться при необходимости: `git reset`

###5. Опишите порядок работы с общим хранилищем VCS.

Получить актуальную версию: `git pull`

Создать ветку для новой функции: `git checkout -b feature`

Внести изменения и закоммитить: `git commit`

Отправить изменения: `git push`

Создать Pull Request для слияния

###6. Каковы основные задачи, решаемые инструментальным средством `git`?

Отслеживание изменений в файлах

Возможность отката к любой версии

Поддержка ветвления и слияния

Распределённая работа над проектом

Совместная работа нескольких разработчиков

###7. Назовите и дайте краткую характеристику командам `git`. Команда Назначение `git init` Создание репозитория `git add` Добавление файлов в индекс `git commit` Фиксация изменений `git status` Просмотр состояния `git diff` Просмотр изменений `git log` Просмотр истории `git branch` Работа с ветками `git checkout` Переключение веток `git merge` Слияние веток `git push` Отправка изменений `git pull` Получение изменений `git clone` Клонирование репозитория

###8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

Локальный репозиторий: `bash`

`git init` `echo "Hello" > file.txt` `git add file.txt` `git commit -m "First commit"`

Удалённый репозиторий: `bash`

`git remote add origin git@github.com:user/repo.git` `git push -u origin main`

###9. Что такое и зачем могут быть нужны ветви (branches)?

Ветви позволяют разрабатывать несколько функций параллельно, не влияя на основную ветку (`main/master`). Это полезно для:

Разработки новых функций

Исправления ошибок

Экспериментов

Работы над разными версиями проекта

###10. Как и зачем можно игнорировать некоторые файлы при commit?

Файл .gitignore содержит шаблоны файлов и каталогов, которые Git не должен отслеживать. Это нужно для:

Временных файлов редакторов

Объектных и бинарных файлов

Конфигурационных файлов с паролями

Логов и кэша

##Вывод

В ходе выполнения лабораторной работы была успешно выполнена первоначальная настройка Git: установлены Git и GitHub CLI, созданы SSH и PGP ключи, настроена подпись коммитов, создано рабочее пространство для курса на основе шаблона. Все изменения загружены на GitHub. Цель работы достигнута. ##Библиография

Dash, P. Getting Started with Oracle VM VirtualBox. – 2013.

Vugt, S. van. Red Hat RHCSA/RHCE 7 cert guide. – 2016.

Робачевский, А. Операционная система UNIX. – 2010.

Немет, Э. Unix и Linux. – 2014.

Robbins, A. Bash Pocket Reference. – 2016.